Nüfus ve Sosyal Araştırmalar Enstitüsü - Yapay Zeka Modülü : Makine Öğrenmesi - 5

İlgili notlar hazırlanırken **Sayın Ercan Atam** hocamızın notlarından faydalanılmıştır. Verdiği eğitim ve güzel anlatımı için teşekkürlerimi sunarım.

BÖLÜM 1: SINIFLANDIRMA (CLASSIFICATION) VE DOĞRUSAL MODELLER

Makine öğrenmesi literatüründe sınıflandırma, bir veri örneğinin öznitelik (feature) kümesi incelenerek, bu örneğin önceden tanımlanmış kategorik sınıflardan birine atanması sürecidir. Bu disiplinin temel amacı, eğitim verileri üzerinden bir karar kuralı inşa etmek ve bu kuralı daha önce sisteme dahil edilmemiş test verileri üzerinde yüksek tahmin başarısıyla uygulamaktır.

Örneğin, bir bankacılık sisteminde müşterinin kredi başvuru verileri modele girer ve model bu verileri "Onayla" veya "Reddet" şeklinde iki sınıfa ayırır. Karmaşık doğrusal olmayan yaklaşımlar mevcut olsa da, doğrusal modeller (linear models) işlem maliyetlerinin düşüklüğü ve şeffaflıkları nedeniyle sınıflandırma problemlerinde referans model (baseline) olarak kabul edilir ve ilk denenecek model olmalıdır.

Ekstra Bilgi: Model Yorumlanabilirliği (Interpretability) Karmaşık yapay sinir ağları ve derin öğrenme modelleri çok yüksek doğruluk oranları sunmalarına karşın, "kara kutu" (black box) doğaları gereği alınan kararların içyüzünü açıklayamazlar. Doğrusal modeller ise şeffaf yapıları sayesinde hangi özniteliğin karara ne ölçüde etki ettiğini katsayılar üzerinden net bir şekilde ortaya koyar. Bu nedenle, kararların hukuki ve mantıksal olarak gerekçelendirilmesinin zorunlu olduğu finans ve sağlık gibi regülasyona tabi sektörlerde, yorumlanabilirliği yüksek olan doğrusal modeller öncelikli olarak tercih edilmektedir.

1.1 Sınıflandırma Probleminin Matematiksel Modellenmesi

1.1.1 Doğrusal Sinyal Mekanizması

Doğrusal sınıflandırıcının kalbinde "Doğrusal Sinyal" (s) kavramı yatar. Bu sinyal, girdi vektörü ile ağırlık parametrelerinin skaler çarpımı sonucunda elde edilir:

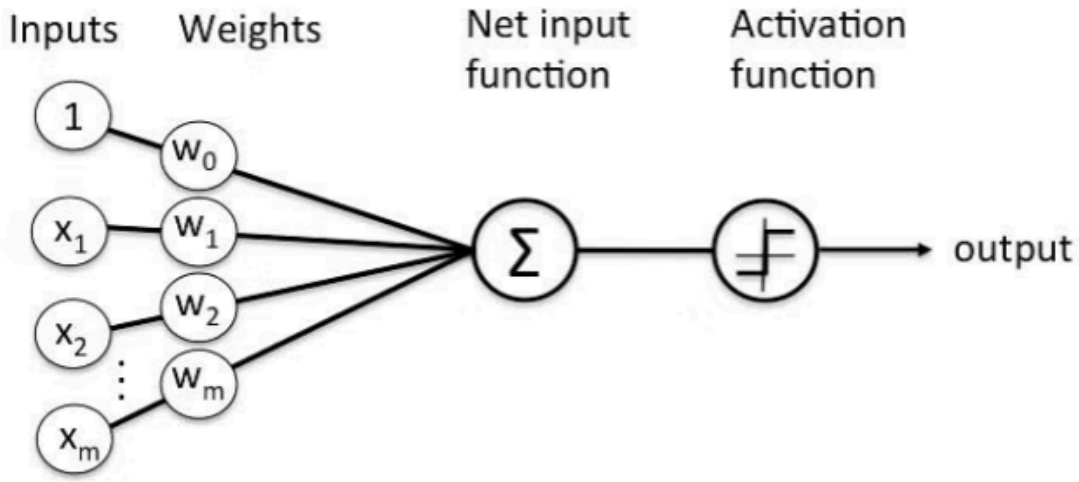
$$s = \mathbf{w}^T\mathbf{x} = w_0 + w_1x_1 + \cdots + w_dx_d$$

Bu formülasyon iki temel perspektif sunar:

1. **Girdilere ($\mathbf{x}$) Göre Doğrusallık**: Geometrik olarak, veriyi öznitelik uzayında ikiye bölen bir hiperdüzlemi (hyperplane separator) veya doğrusal bir ayırıcıyı tanımlar.
2. **Ağırlıklara ($\mathbf{w}$) Göre Doğrusallık**: Algoritmik olarak, öğrenme süreçlerinde modelin optimizasyon işlemlerini matematiksel olarak kolaylaştırır ve hesaplanabilir kılar.

Modelin sabit terimi olan w_0 (bias) değerini matris işlemlerine dahil etmek amacıyla, girdi vektörü $\mathbf{x}$'in başına 1 değeri eklenerek genişletilmiş (augmented) bir vektör yapısı kurulur: $\mathbf{x} \in \{1\} \times \mathbb{R}^d$.

1.1.2 Aktivasyon ve Karar Süzgeci



Hesaplanan doğrusal sinyal (s), karar mekanizmasının sadece nümerik temelidir ve nihai bir karara dönüşmek için bir aktivasyon fonksiyonuna iletilmesi gerekir. Doğrusal sınıflandırmada bu işlev genellikle işaret (sign) fonksiyonu ile yerine getirilir:

$$\text{Nihai Karar} = \text{sign}(\mathbf{w}^T\mathbf{x}) \in \{-1, +1\}$$

Bu işlem, veriyi hesaplanan sinyalin pozitif veya negatif olmasına göre kesin bir şekilde iki karşıt kümeden birine atar.

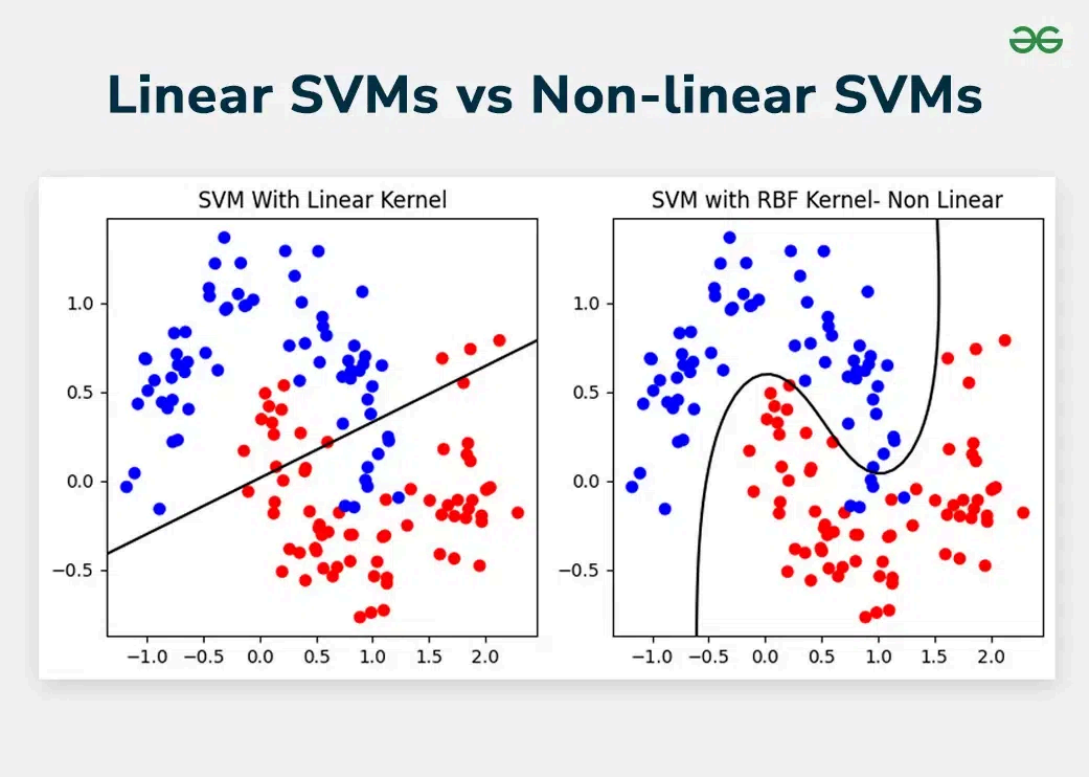
1.2 Ayrılabilirlik Analizi ve Öğrenme Algoritmaları

Veri noktalarının uzaydaki dağılım özellikleri, modelin başarısını ve seçilecek algoritmayı doğrudan belirler:

1. **Doğrusal Ayrılabilirlik (Linear Separability)**: İki sınıfın temsil edildiği noktalar arasında hiçbir hata olmaksızın düz bir ayraç çekilebiliyorsa, Perceptron Öğrenme Algoritması (PLA) kullanılır. Bu algoritma, veri doğrusal olarak ayrılabildiği sürece hatasız bir şekilde çalışır.
2. **Yaklaşık Ayrılabilirlik ve Pocket Algoritması**: Gerçek dünya verileri genellikle iç içe geçmiş durumdadır ve kusursuz bir ayrım nadiren mümkündür. PLA'nın hatalı noktalarda kontrolsüz dalgalanmalar yaşamasını engellemek için Pocket algoritması geliştirilmiştir.

Bu algoritma, iterasyonlar boyunca en düşük hata oranına sahip ağırlık setini cebinde (bellekte) saklayarak istikrarlı bir çözüm sunar.

3. **Doğrusal Olmayan Ayrılabilirlik**: Verilerin dairesel veya karmaşık bir dağılım gösterdiği durumlarda doğrusal bir çizgi işlevsiz kalır. Bu tür veriler, doğrusal olmayan dönüşümlerle analiz edilir.



Ekstra Bilgi: Çekirdek Hilesi (Kernel Trick) ve Boyutsal Yükseltme İki boyutlu uzayda iç içe geçmiş veya dairesel bir yapı sergileyen veri kümeleri doğrusal bir çizgi ile ayrılamaz. Bu problemin aşılması için veri, matematiksel transformasyonlar (örneğin $z_1 = (x_1 + 0.05)^2$) aracılığıyla daha yüksek boyutlu bir uzaya (örneğin üçüncü boyuta) yansıtılır. Düşük boyutta doğrusal olarak ayrılamayan bu yapı, yüksek boyutta araya düz bir plaka (hiperdüzlem) yerleştirilerek doğrusal hale getirilir. Literatürde "Çekirdek Hilesi" (Kernel Trick) olarak adlandırılan bu zarif matematiksel çözüm, Destek Vektör Makineleri (SVM) algoritmalarının temelini oluşturur.

1.3 Vaka Çalışması: USPS El Yazısı Rakam Tanıma

ABD Posta Servisi'nin (USPS) el yazısı karakter tanıma veri seti üzerinden, boyut indirgeme ve doğrusal sınıflandırma ilkeleri incelenmiştir.

- **Boyutsallık ve Öznitelik Çıkarımı**: 16x16 piksellik görüntüler 256 boyutlu bir ham veri uzayı oluşturur. Sadece "1" ve "5" rakamlarını ayırmak hedeflendiğinde, 256 pikselle tek tek ilgilenmek yerine iki temel makro öznitelik (feature) türetilir: "Simetri" ve "Ortalama Yoğunluk" (piksel/mürekkep yoğunluğu).
- **Veri Analizi**: "1" rakamının yüksek simetri değerine ve düşük piksel yoğunluğuna sahip olduğu; "5" rakamının ise asimetrik yapısıyla daha fazla mürekkep yoğunluğu sergilediği

saptanır.

- **Model Performansı**: Bu iki özelliğe indirgenen veri üzerinde, Pocket algoritması eğitildiğinde eğitim hatasının (E_{in}) iterasyonlar ilerledikçe merdiven basamağı şeklinde istikrarlı bir düşüş gösterdiği; PLA'nın ise hata grafiğinde inip çıkan istikrarsız bir yapı çizdiği tespit edilmiştir.

1.4 İkili ve Çoklu Sınıflandırma Ayrımı

Sınıflandırma görevleri, hedeflenen çıktı sayısına göre ikiye ayrılır:

- **İkili Sınıflandırma (Binary)**: Veriyi sadece iki karşıt kümeden birine dahil eder (Onaylandı/Reddedildi, 1/5 gibi).
- **Çoklu Sınıflandırma (Multi-class)**: Veriyi ikiden fazla kategori arasından sınıflandırır (Tüm rakamları 0'dan 9'a kadar tanıma gibi).

Ekstra Bilgi: Çoklu Sınıflandırma Stratejileri Teorik olarak en temel sınıflandırma algoritmaları doğası gereği ikili (binary) formda çalışır. Algoritmaların çok sınıflı problemlere uyarlanması gerektiğinde, genellikle "Bire Karşı Hepsi" (One-vs-All) veya "Bire Karşı Bir" (One-vs-One) stratejileri benimsenir. Bu yaklaşımlarda problem, eşzamanlı çalışan birden fazla alt ikili sınıflandırma problemine bölünerek çözülür. Derin öğrenme mimarilerinde ise bu işlem çıktı katmanında Softmax aktivasyon fonksiyonu kullanılarak olasılıksal bir dağılıma dönüştürülür.

BÖLÜM 2: KÜMELEME (CLUSTERING) VE DENETİMSİZ ÖĞRENME

Makine öğrenmesinde sınıflandırma algoritmaları etiketli (hedef değişkeni bilinen) verilerle çalışırken, kümeleme (clustering) algoritmaları etiketlenmemiş verilerdeki gizli yapıları ve doğal gruplaşmaları keşfetmeyi amaçlayan denetimsiz öğrenme (unsupervised learning) yöntemleridir.

Bir küme (cluster), analiz edilen veri uzayı içerisinde birbirine istatistiksel veya geometrik olarak yüksek oranda benzerlik gösteren veri noktalarından oluşan bir topluluktur.

Matematiksel bir k-kümeleme (k-clustering) işlemi, $\mathcal{D}$ veri setinin k adet alt kümeye ($\mathcal{S}_1, \mathcal{S}_2, \ldots, \mathcal{S}_k$) bölünmesi olarak tanımlanır. Bu bölme işlemi şu iki temel topolojik şartı sağlamalıdır:

1. Tüm kümelerin birleşimi ana veri setini oluşturmalıdır: $\cup_{j=1}^{k} \mathcal{S}_j = \mathcal{D}$
2. Kümeler birbirini dışlamalıdır (kesişimleri boş küme olmalıdır): $\mathcal{S}_i \cap \mathcal{S}_j = \emptyset$ (her $i \neq j$ için)

Her $\mathcal{S}_j$ kümesi, o kümedeki noktaların uzaysal konumlarını temsil eden bir ağırlık merkezine (μ_j) sahiptir.

2.1 Optimum Kümelemenin Geometrik Karakteristiği

Başarılı bir kümeleme algoritmasının veri uzayında sağlaması beklenen iki temel istatistiksel özellik bulunmaktadır:

1. **Küme İçi Yayılımın (Within-cluster spread) Minimizasyonu:** Bir küme içindeki veri noktaları birbirlerine ve ait oldukları merkeze uzaysal olarak çok yakın olmalıdır. Bu, kümenin kendi içindeki varyansının (homojenliğinin) yüksek olduğunu gösterir.
2. **Kümeler Arası Yayılımın (Between-cluster spread) Maksimizasyonu:** Farklı kümelerin merkezleri ve sınırları birbirinden olabildiğince uzak ve ayırt edilebilir olmalıdır.

Ekstra Bilgi: Optimizasyon Önceliği Teorik olarak her iki koşul da önemli olmakla birlikte, pratikte algoritmalar yalnızca birinci koşulu (küme içi yayılımı) minimize etmeye odaklanacak şekilde tasarlanır. Çünkü uzaysal kısıtlar gereği, veri noktalarının kendi merkezlerine olan mesafeleri (hata) minimize edildiğinde, kümeler arası ayrışma da matematiksel olarak doğal bir sonuç (yan ürün) biçiminde maksimize edilmiş olur.

Maliyet Fonksiyonunun Formülasyonu

Kümeleme işleminin başarısı, tüm veri noktalarının ait oldukları küme merkezine olan uzaklıklarının karesel toplamı ile ölçülür.

j kümesindeki lokal hata:

$$E_j = \sum_{x^n \in \mathcal{S}_j} ||x^n - \mu_j||^2$$

Tüm sistemin (k-ortalama) toplam hata fonksiyonu (E_{in}):

$$E_{in}(\mathcal{S}_1, \ldots, \mathcal{S}_k; \mu_1, \ldots, \mu_k) = \sum_{n=1}^{N} ||x^n - \mu(x^n)||^2$$

Burada $\mu(x^n)$, x^n noktasının atandığı kümenin merkez koordinatıdır. Kümeleme probleminin nihai amacı, bu E_{in} maliyet fonksiyonunu global veya lokale en yakın minimum seviyeye çekecek $\mathcal{S}_j$ küme atamalarını ve μ_j merkez koordinatlarını eşzamanlı olarak bulmaktır.

2.2 K-Means (K-Ortalama) ve Lloyd Algoritması

$\mathcal{S}_j$ (kümeler) ve μ_j (merkezler) değerlerini aynı anda optimize etmek hesaplama karmaşıklığı açısından NP-Zor (NP-Hard) bir problemdir. Ancak, problemin doğasındaki şu iki analitik gözlem, iteratif bir çözüm yolu sunar:

- **Gözlem 1:** Eğer merkez noktalarının (μ_j) yerleri sabit kabul edilirse, optimum kümeleri oluşturmak basittir; her veri noktası salt öklid uzaklığına bakılarak en yakın merkezin kümesine atanır.
- **Gözlem 2:** Eğer kümeler ($\mathcal{S}_j$) sabit kabul edilirse, bu kümeyi en az hatayla temsil edecek yeni merkezi bulmak basittir; kümedeki tüm veri noktalarının aritmetik ortalaması (ağırlık

merkezi) alınır:

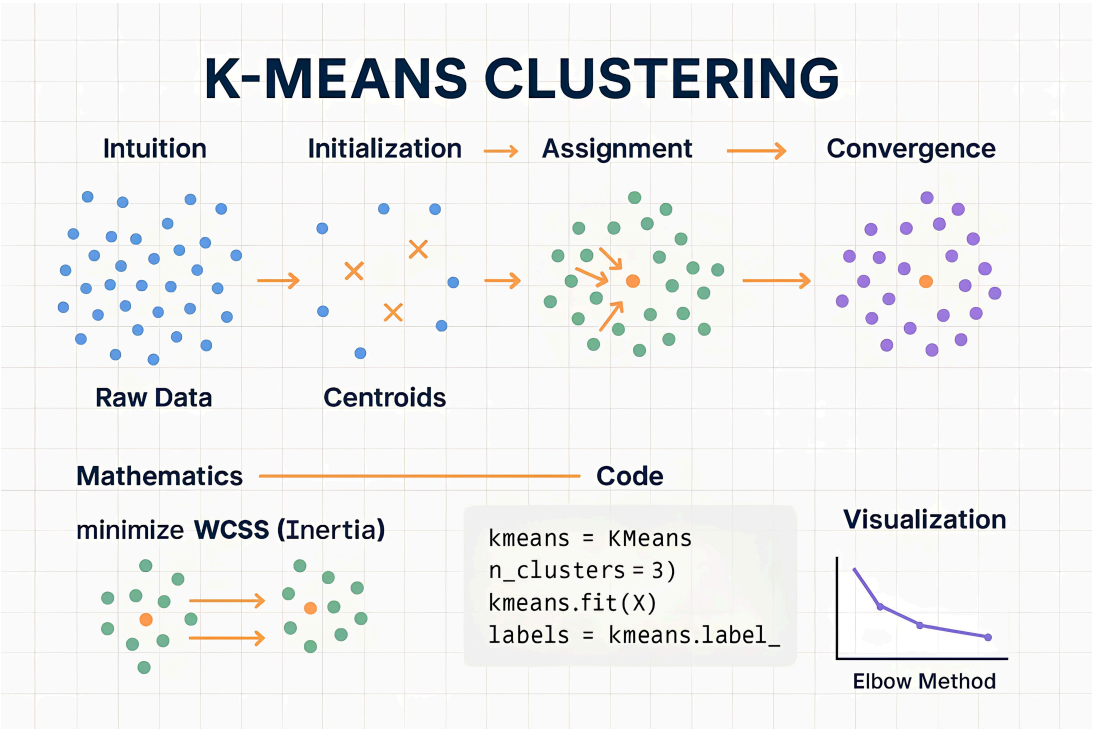
$$\mu_j = \frac{1}{|\mathcal{S}_j|} \sum_{x^n \in \mathcal{S}_j} x^n$$

Bu gözlemlerden yola çıkarak geliştirilen ve literatürde **Lloyd Algoritması** olarak bilinen K-Means yöntemi, şu adımları izler:

1. **Başlatma (Initialization):** Veri uzayında birbirinden olabildiğince ayrık k adet rastgele merkez noktası (μ_j) belirlenir.
2. **Atama Adımı (Update Clusters):** Veri setindeki her bir x^n noktası, kendisine en yakın merkeze atanarak $\mathcal{S}_j$ kümeleri güncellenir.

$$\mathcal{S}_j \leftarrow \{x^n : ||x^n - \mu_j|| \leq ||x^n - \mu_l|| \quad \text{for } l = 1, \ldots, k\}$$

3. **Merkez Güncelleme Adımı (Update Centers):** Yeniden oluşturulan $\mathcal{S}_j$ kümelerindeki noktaların aritmetik ortalaması alınarak μ_j merkezleri uzayda yeni konumlarına kaydırılır.
4. **Yakınsama (Convergence):** E_{in} (toplam hata) değerindeki düşüş durana veya merkez noktalarının koordinatları sabitlenene kadar 2. ve 3. adımlar tekrarlanır.



2.3 Algoritmanın Yakınsama (Convergence) Özellikleri ve Kısıtları

Lloyd algoritması çalıştırıldığında, her bir atama ve merkez kaydırma adımında E_{in} (hata) fonksiyonu istisnasız olarak ya azalır ya da sabit kalır; asla artmaz.

- **Sonlu Sürede Yakınsama Garantisi:** Veri noktalarının k adet kümeye atanabileceği kombinasyon sayısı üstel (exponential) derecede büyük olsa da, matematiksel olarak

sonlu bir sayıdır. Hata fonksiyonu sonsuza kadar azalamayacağı için, algoritma teorik olarak sonlu bir sürede kesinlikle durur (yakınsar).

- **Yerel Minimum (Local Optima) Problemi:** Algoritmanın en büyük zaafı, ardışık (alternatifleyen) bir optimizasyon tekniği olmasıdır. Bu durum, algoritmanın her zaman en mükemmel (global) çözüme değil, başlangıç noktalarına bağlı olarak tatminkar bir yerel minimuma (local optima) yakınsamasıyla sonuçlanabilir.

Ekstra Bilgi: Başlatma Optimizasyonu (K-Means++) K-Means algoritmasının standart Lloyd varyasyonunda merkezler tamamen rastgele atanır, bu da şans eseri kötü bir başlangıç yapıldığında kümelerin yanlış ayrışmasına yol açabilir. Modern makine öğrenmesi kütüphanelerinde (örneğin Scikit-Learn) bu sorunu aşmak için "K-Means++" adı verilen zekice bir başlatma algoritması endüstri standardı olarak kullanılır. K-Means++, ilk merkezi rastgele seçtikten sonra, ardışık merkezleri mevcut merkezlere en uzak olan veri noktaları arasından olasılıksal bir ağırlıklandırma ile seçer. Bu önlem, yerel minimuma takılma riskini büyük ölçüde elimine eder ve algoritmanın yakınsama hızını artırır.

BÖLÜM 3: REGRESYONDA DÜZENLİLEŞTİRME (REGULARIZATION) VE AŞIRI UYUM YÖNETİMİ

Makine öğrenmesi modelleri eğitilirken karşılaşılan en temel sorunlardan biri, modelin eğitim verisine çok yüksek bir esneklikle uyum sağlayıp verideki rastgele gürültüleri bile ezberlemesi durumudur. "Aşırı Uyum" (Overfitting) olarak adlandırılan bu senaryoda model, eğitim verisinde çok düşük hata oranları verirken, daha önce hiç görmediği test verisinde başarısız olur.

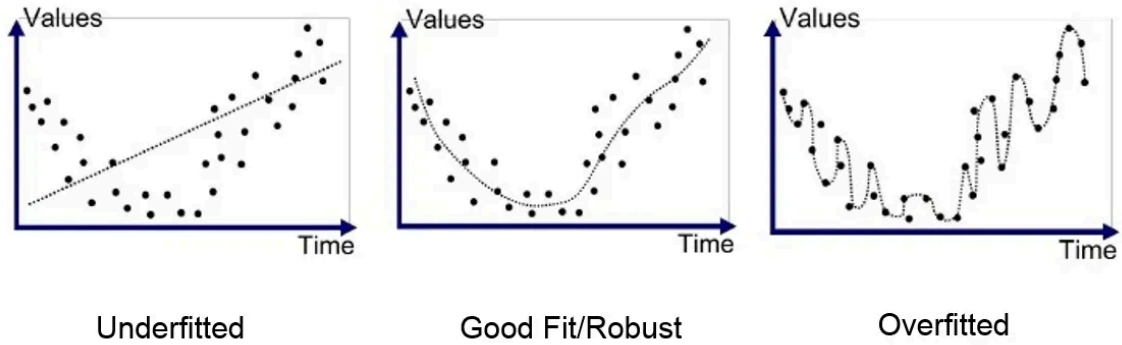
Düzenlileştirme (Regularization), aşırı uyumu ortadan kaldırmak ya da etkisini asgari düzeye indirmek amacıyla, modelin matematiksel esnekliğini bilinçli bir şekilde sınırlayan ve kontrol altına alan bir optimizasyon tekniğidir.

3.1 Düzenlileştirmenin Temel Felsefesi

Parametrik modeller bağlamında düzenlileştirmenin temel kuralı şu felsefeye dayanır: "*Eğitim verisi bizi kesin ve güçlü bir şekilde aksi yönde ikna etmedikçe, ağırlık parametreleri vektörünü ($\mathbf{w}$) niceliksel olarak olabildiğince küçük tut.*" Başka bir deyişle, küçük parametre değerlerine sahip daha basit bir model ile büyük parametrelere sahip karmaşık bir model veriyi benzer oranlarda açıklayabiliyorsa, her zaman küçük parametreli basit model tercih edilmelidir (Occam'ın Usturası prensibi).

Düzenlileştirme algoritmik olarak iki ana kategoriye ayrılır: Açık (Explicit) ve Örtük (Implicit) Düzenlileştirme.

3.2 Açık (Explicit) Düzenlileştirme Mekanizmaları



Açık düzenlileştirme, modelin eğitim aşamasında minimize etmeye çalıştığı maliyet fonksiyonunun (cost function) içerisine matematiksel bir "ceza" (penalty) terimi eklenerek uygulanır. Genel formülasyonu şu şekildedir:

$$\hat{w} = \arg\min_{w} \left[J(w; \mathcal{D}) + R(w; \lambda) \right]$$

Bu denklem, birbirini dengeleyen üç ana bileşenden oluşur:

1. $J(w; \mathcal{D})$ **(Maliyet Fonksiyonu)**: Modelin eğitim verisine ($\mathcal{D}$) ne kadar iyi uyum sağladığını ölçer (Hatayı temsil eder).
2. $R(w; \lambda)$ **(Düzenlileştirme Terimi)**: Parametre değerlerinin küçük kalmasını teşvik eden ceza fonksiyonudur.
3. λ **(Lambda - Takas Parametresi)**: Modelin veriye uyumu ile parametrelerin küçüklüğü arasındaki ödünleşimi (trade-off) belirleyen hiperparametredir.

Açık düzenlileştirme, uygulanan matematiksel cezanın türüne göre iki özel metoda ayrılır:

3.2.1 L^2 Düzenlileştirme (Ridge Regresyon)

Bu yöntemde parametrelerin karelerinin toplamı ceza terimi olarak fonksiyona eklenir. Doğrusal regresyona uygulandığında Ridge Regresyon adını alır:

$$J_{RR}(w) = \frac{1}{N} \sum_{i=1}^{N} (w^T x^i - y^i)^2 + \lambda \sum_{j=1}^{d} w_j^2$$

- **Çalışma Prensibi:** Formül, tüm ağırlık parametrelerini (w_j) sıfıra doğru iter ancak hiçbir zaman tam olarak sıfır yapmaz. Ağırlıkları orantısal olarak küçültür.
- **Matris Çözümü Avantajı:** Klasik doğrusal regresyonda matris tersinin alınamaması (singülerlik) sorununu çözer. Hata fonksiyonu türetilip sıfıra eşitlendiğinde kapalı form (closed-form) bir matris çözümü sunar: $w_{lin} = (X^T X + N\lambda I_d)^{-1} X^T y$. Denklemdeki I_d (Birim Matris) ve λ sayesinde matris her zaman terslenebilir (invertible) hale gelir.

3.2.2 L^1 Düzenlileştirme (LASSO)

Bu yöntemde parametrelerin mutlak değerlerinin toplamı ceza terimi olarak fonksiyona eklenir (Least Absolute Shrinkage and Selection Operator).

$$J_{LASSO}(w) = \frac{1}{N}\sum_{i=1}^{N}(w^T x^i - y^i)^2 + \lambda \sum_{j=1}^{d} |w_j|$$

- **Çalışma Prensibi ve Seyreklik (Sparsity):** L^2'nin aksine, LASSO optimizasyon sürecinde modele katkısı az olan özniteliklerin (parametrelerin) ağırlıklarını **tam olarak sıfıra eşitler**.
- **Öznitelik Seçimi:** Geriye sadece gerçekten önemli özellikleri bıraktığı için, aynı zamanda otomatik bir Girdi/Öznitelik Seçimi (Feature Selection) mekanizması olarak da çalışır.
- Mutlak değer fonksiyonunun türevi her noktada alınamadığı için kapalı form matris çözümü yoktur; nümörik optimizasyon (Örn: Gradient Descent) ile çözülmesi gerekir.

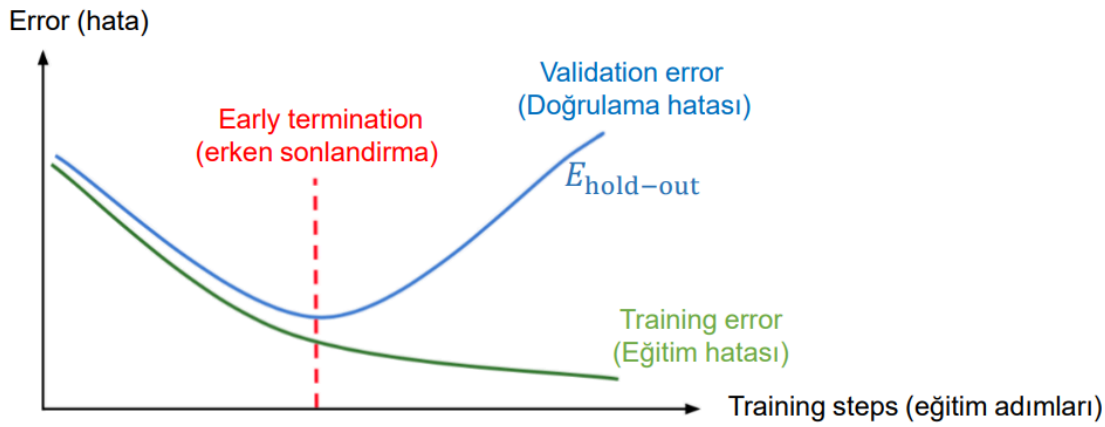
Ekstra Bilgi: Elastik Ağ (Elastic Net) Düzenlileştirmesi Gerçek dünya verilerinde genellikle özellikler (features) arasında yüksek korelasyon bulunur. LASSO, birbiriyle ilişkili özelliklerden rastgele birini seçip diğerlerini sıfırlama eğilimindedir. Hem L^1'in öznitelik seçme (sıfırlama) yeteneğini, hem de L^2'nin katsayıları stabilize etme gücünü aynı anda kullanmak isteyen araştırmacılar "Elastic Net" modelini geliştirmiştir. Bu modelde her iki ceza terimi eşzamanlı olarak fonksiyona dahil edilir: $R(w;\lambda) = \lambda_1||w||_1 + \lambda_2||w||_2^2$.

3.3 Takas Parametresi (Lambda - λ) ve Optimizasyon

Düzenlileştirme denklemindeki λ parametresinin büyüklüğü, modelin kaderini doğrudan belirler:

- $\lambda = 0$ **(Aşırı Uyum):** Hiçbir düzenlileştirme uygulanmaz. Model aşırı karmaşık hale gelir ve eğitim verisini ezberler.
- **Optimum λ:** Modelin esnekliği dengelenir; verinin asıl trendi yakalanır ve test başarısı artar.
- **Çok Yüksek λ (Eksik Uyum / Underfitting):** Ceza o kadar büyük olur ki, model hiçbir parametreyi artıramaz. Model aşırı basit, adeta düz bir çizgi haline gelir ve veriyi öğrenemez.

3.4 Örtük (Implicit) Düzenlileştirme: Erken Durdurma (Early Stopping)



Erken durdurma, maliyet fonksiyonuna harici bir matematiksel ceza terimi eklemeden gerçekleştirilen bir örtük düzenlileştirme tekniğidir.

Nasıl Uygulanır? Yinelemeli (iterative) optimizasyon algoritmalarında eğitim süreci boyunca eğitim hatası (Training Error) sürekli olarak azalma eğilimindedir. Modelin ezberlemeye geçmesini engellemek için, veri setinin bir kısmı önceden **Doğrulama Verisi (Validation / Hold-out Data)** olarak kenara ayrılır.

- Optimizasyonun her t adımında, model bu doğrulama verisi üzerinde test edilir ve $E_{hold-out}$ hatası hesaplanır.
- Doğrulama hatası başlangıçta eğitim hatasıyla birlikte azalır. Ancak model veriyi ezberlemeye başladığı "kırılma noktasında", eğitim hatası düşmeye devam etse dahi doğrulama hatası ($E_{hold-out}$) yükselişe geçer (grafikte bir U şekli çizer).
- Algoritma, doğrulama hatasının tam olarak minimuma ulaştığı ve yükselmeye başladığı bu dip noktada **vaktinden önce (erken) sonlandırılır**.

Ekstra Bilgi: Derin Öğrenmede Endüstri Standardı Erken Durdurma, kulağa eğitim sürecini yarıda kesmek gibi sezgiye aykırı (counter-intuitive) bir işlem gibi gelse de, günümüzde Milyarlarca parametreye sahip Büyük Dil Modellerinin (LLM) ve Derin Yapay Sinir Ağlarının aşırı uyumunu (overfitting) engellemek için en çok güvenilen ve en yaygın kullanılan endüstri standardı haline gelmiştir. Bu yöntem genellikle "Model Checkpointing" ile birlikte kullanılır; model hata grafiğinin en dibinde olduğu ağırlık matrislerini kaydeder ve süreci bitirir.

BÖLÜM 4: JUPYTERLAB PRATİK VAKA ÇALIŞMASI (EL YAZISI RAKAM ANALİZİ)

Bu bölümde, teorik derslerimizde işlediğimiz Sınıflandırma, L1/L2 Düzenlileştirme ve K-Means Kümeleme konularını tek bir veri seti (`digits`) üzerinde endüstri standartlarında koda dökeceğiz.

Hücre 1: Gerekli Kütüphanelerin ve Veri Setinin Yüklenmesi

İlk adımda çevremizi hazırlıyor ve tıpkı dersteki gibi 16×16 yerine 8×8 piksellik (64 boyutlu) el yazısı rakam verisini sisteme dahil ediyoruz.

In [8]:
```
# Temel Kütüphaneler
import numpy as np
import matplotlib.pyplot as plt
import warnings
warnings.filterwarnings('ignore')
# Makine Öğrenmesi Kütüphaneleri (Scikit-Learn)
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.cluster import KMeans
from sklearn.metrics import accuracy_score

# 1. Veriyi Yükleme (Öznitelikler: X, Etiketler: y)
digits = load_digits()
X = digits.data   # 64 boyutlu girdi vektörü (Pikseller)
y = digits.target # 0-9 arası rakam etiketleri

# 2. Veriyi Eğitim (%80) ve Test (%20) Olarak Bölme
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_sta

print(f"Toplam Veri Sayısı: {X.shape[0]}")
print(f"Eğitim Verisi Boyutu: {X_train.shape}, Test Verisi Boyutu: {X_test.shape}")

# 1. L2 Düzenlileştirme (Ridge Benzeri - Tüm ağırlıkları küçültür)
model_l2 = LogisticRegression(penalty='l2', C=0.1, max_iter=5000)
model_l2.fit(X_train, y_train)

# 2. L1 Düzenlileştirme (Lasso Benzeri - Seyreklik yaratır, gereksiz pikselleri sıf
model_l1 = LogisticRegression(penalty='l1', solver='liblinear', C=0.1, max_iter=500
model_l1.fit(X_train, y_train)

# Tahmin ve Başarı Ölçümü
y_pred_l2 = model_l2.predict(X_test)
y_pred_l1 = model_l1.predict(X_test)

print(f"L2 Düzenlileştirme Doğruluk Oranı: {accuracy_score(y_test, y_pred_l2):.4f}"
print(f"L1 Düzenlileştirme Doğruluk Oranı: {accuracy_score(y_test, y_pred_l1):.4f}"

# L1'in Seyreklik (Sparsity) Etkisini İspatlama
aktif_parametre_l2 = np.sum(model_l2.coef_ != 0)
aktif_parametre_l1 = np.sum(model_l1.coef_ != 0)

print(f"L2 Modelinde Sıfırdan Farklı Parametre Sayısı: {aktif_parametre_l2}")
print(f"L1 Modelinde Sıfırdan Farklı Parametre Sayısı: {aktif_parametre_l1} (Öznite

# 1. K-Means Modelini Tanımlama (K=10 Küme, K-Means++ başlatma algoritması ile)
# n_init=10: Algoritmanın farklı rastgele merkezlerle 10 kez çalışıp en iyi (en düş
kmeans = KMeans(n_clusters=10, init='k-means++', n_init=10, random_state=42)

# 2. Modeli Eğitme (Sadece X veriliyor, y etiketleri saklanıyor!)
kmeans.fit(X)
```

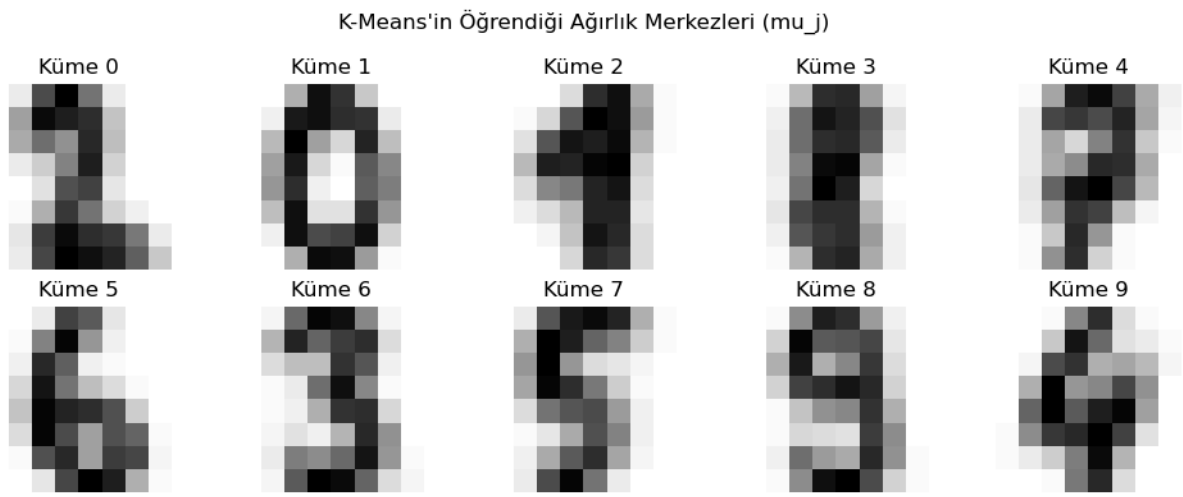
```python
# 3. Öğrenilen Kümeleri ve Merkezleri Çekme
kume_merkezleri = kmeans.cluster_centers_  # 10 adet ağırlık merkezi (mu_j)
nokta_atamalari = kmeans.labels_           # Hangi noktanın hangi kümeye atandığı

print(f"Küme Merkezleri Matris Boyutu: {kume_merkezleri.shape}") # (10, 64) çıkacak
print(f"Küme İçi Yayılım Hatası (E_in - Inertia): {kmeans.inertia_:.2f}")

# 4. Küme Merkezlerini Görselleştirme (Algoritmanın kafasındaki "Tipik" rakamlar)
fig, ax = plt.subplots(2, 5, figsize=(10, 4))
for i, merkez in enumerate(kume_merkezleri):
    # 64 boyutlu vektörü tekrar 8x8 görüntü formatına çeviriyoruz
    merkez_resmi = merkez.reshape(8, 8)
    ax[i//5, i%5].imshow(merkez_resmi, cmap='binary')
    ax[i//5, i%5].set_title(f"Küme {i}")
    ax[i//5, i%5].axis('off')

plt.suptitle("K-Means'in Öğrendiği Ağırlık Merkezleri (mu_j)")
plt.tight_layout()
plt.show()
```

```
Toplam Veri Sayısı: 1797
Eğitim Verisi Boyutu: (1437, 64), Test Verisi Boyutu: (360, 64)
L2 Düzenlileştirme Doğruluk Oranı: 0.9722
L1 Düzenlileştirme Doğruluk Oranı: 0.9611
L2 Modelinde Sıfırdan Farklı Parametre Sayısı: 610
L1 Modelinde Sıfırdan Farklı Parametre Sayısı: 251 (Öznitelik Seçimi Yapıldı!)
Küme Merkezleri Matris Boyutu: (10, 64)
Küme İçi Yayılım Hatası (E_in - Inertia): 1165256.30
```



-Memreu